YARA: A Pattern-Based Malware Detection

Explaining core functionalities

Name: Md. Farhan Mahtab
StudentID: 1805096

Name: Shahriar Ferdoush Sifat
StudentID: 1805101

CSE 406 Computer Security
Department Of CSE,
BUET

14th September 2023

Contents

1 Introduction

YARA is an open-source tool and a popular pattern-matching tool used primarily for malware detection and classification. It allows security researchers, malware analysts, and cybersecurity professionals to create custom rules (YARA rules) to identify and classify files or processes based on specific patterns or characteristics.

A brief overview of how YARA works as a Malware detection and attack prevention system is given below:

- **Rule Creation:** Security experts create YARA rules using a simple and flexible syntax. These rules define patterns, strings, or conditions that describe the characteristics of malware or other files they want to detect.
- **Pattern Matching:** YARA scans files, memory, or processes for matches to these rules. When it finds a match, it can trigger various actions, such as alerting the user, logging the event, or taking other specified actions.
- **Customization:** ARA is highly customizable. Analysts can create rules based on strings, binary patterns, regular expressions, and more. This flexibility makes it effective for identifying various types of malware, from known to unknown threats.
- **Community Sharing:** The YARA community actively shares and maintains rule sets, making it easier for organizations to leverage the collective knowledge and improve their threat detection capabilities.
- **Integration:** YARA can be integrated into various security tools and workflows, making it an essential part of many cybersecurity setups.

  For Example, integrating YARA with Zeek, a network security monitoring tool, enhances network security by allowing real-time pattern-based malware detection and classification. Zeek passively monitors network traffic, while YARA's custom rules define patterns and conditions to identify potential threats within this traffic. When Zeek detects network activity or files that match YARA rules, it triggers alerts or logs, facilitating rapid incident response and improving overall network security. This integration combines Zeek's network visibility with YARA's malware detection capabilities, providing an adaptable and effective solution for identifying and mitigating network-based threats.

2 Code Review

MakefIle.am: This file

- Defines rules for configuring, compiling, and linking the YARA project. It specifies compiler flags, source files, and dependencies needed to build YARA on various platforms.
- Contains rules for generating code files from .proto files (Protocol Buffers). These generated files are used for specific YARA modules.
- Provides Testing and Debugging Support.

libyara: This directory contains the library funtion. Inside this there are other directories.

- **include:** Contains yara.h which contains all the basic header files inside it. The included headers are then defined inside the yara directory inside the include directory
- **module:** Contains code for all the available modules. For example: we learned about cuckoo module. Code for that available inside cuckoo.c file inside cuckoo folder.
- **proc:** Contains how processes and memory and other staffs are handle for different Operating systems. For example linux.c is for linux.

Also it contains other file directly inside it. For example:

- **grammar.y, hex_grammar.y and re_grammar.y :** Parser for normal string matching, hex matching and regular expression.
- **lexer.l, hex_lexer.l and re_lexer.l :** Lexical analyzers for normal string, hex rules and regular expressions
- **mem.c :** For memory management staffs.
- **Others like rules.c, threads.c etc.**

cli: This directory contains files related to command line interface.

- **args.c:** This file contains functions related to handling commnad line arguments i.e. args_get_short_option.
- **args.h:** This header file declares functions of args.c. Also it has some enum, struct, macros used with functions of args.c.
- **common.c:** This file contains command line argument data type determining functions (is_int, is_float), parsing file name and provides them to compiling api functions(compile_files).
- **common.h:** Declares the functions and data types of common.c.
- **threading.c** Contains all functions related thread creation, mutex lock logic, semaphore logic, thread destruction.
- **threading.h:** Declares the functions and data types of threading.c.

- **yara.c** Contains function related to acquiring rule files from argument and use the rules to inspect given file/files. It also shows various output depending on various flag used in command line arguments.
- **yarac.c:** It contains function relating to compiling rule files and showing the result/error message in command line.

dist: This directory contains files related to building RPM file.

- **yara.spec:** This file contains specification for RPM to build the package.
- **yara-python.spec:** This file contains specification for RPM to build the yara-python.

docs: This directory contains documentaion files of all features, modules, functions.

extra: This directory contains webpage and javascript files providing a codemirror editor for testing yara functionality.

m4: This directory contains single file which is modified bug free version of original acx_pthread.m4.

tests: This directory contains files related testing if the installation was done correctly and all installed module are working correctly.

3 YARA Rules

3.1 Basic Structure

The basic structure of a YARA rule consists of three main components: "Meta," "Strings," and "Condition." The "Meta" section allows provide metadata about the rule, including authorship and descriptions. "Strings" is where variables are decleared to store patterns or keywords you want to search for, enhancing readability and reusability. The "Condition" section defines the criteria for identifying potentially malicious content, utilizing the declared variables and YARA's powerful syntax.

Here is a basic structure of a YARA rule:

```
rule RuleName
{
    % Metadata for the rule
    meta:
        author = "Your Name"
        description = "Description of your YARA rule"
        reference = "Optional reference information"

    % Variables
    strings:
        $myString = "malware"
        $hexPattern = { 48 65 6C 6C 6F }

    % Condition
    condition:
        % Use variables in the condition
        $myString or $hexPattern or regex /malw[0-9]{3}/
}
```

Listing 1: Basic YARA Rule Structure

This is the basic structure.

3.2 First YARA Rule Working

Here is a basic example of an YARA Rule called hello [1]. This hello rule is matched is the "$hello_string" or the "$byte_string" is matches in the file, folder or disk image or whatever is passed in to be scanned with YARA.

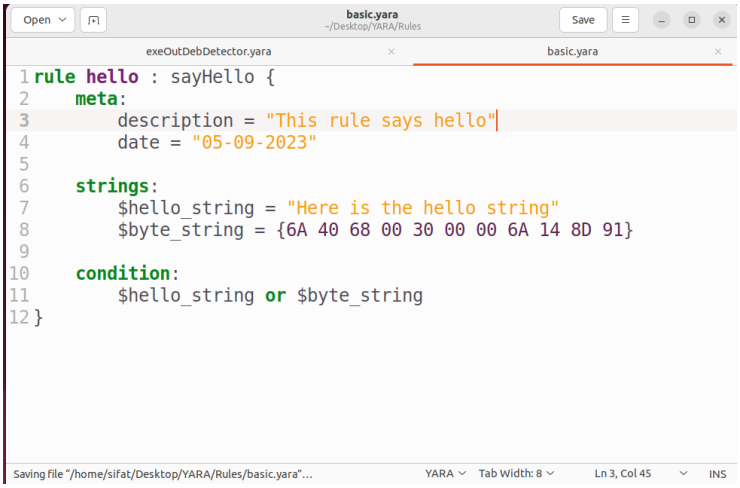

Figure 1: Hello string matching rule

To run YARA rule, We type

$$yara\ file\_name.yara\ what\_is\_to\_be\_scanned$$

Here is a screenshot showing the Hello rule running :

Figure 2: Hello Rule in action

Here running the basic.yara file containing the hello rule, we can see YARA shows hello, which is the rule that is being matched and then the file where it matched. To varify that YARA successfully detected the right file we can see lateron the content of the hello.virus file.

3.3 Other Examples

Now for some better understanding lets work with some real files. Here are two rules that are written to detect exe files and out or deb file respectively.

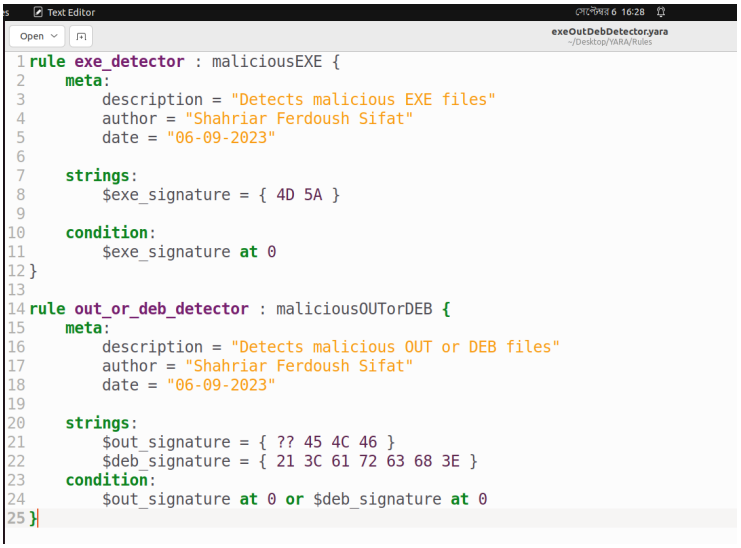

Figure 3: Executable Files detection Rules

In exe_detector there is a Hex sequence declared { 4D 5A}. Then if the string is found at the beginning of the files bytestream then this rule is matched and we detect that the selected file is an exe file.

Here we have opened an .exe file in a hex editor. we can see the exe signature at the beginning of the hex stream.

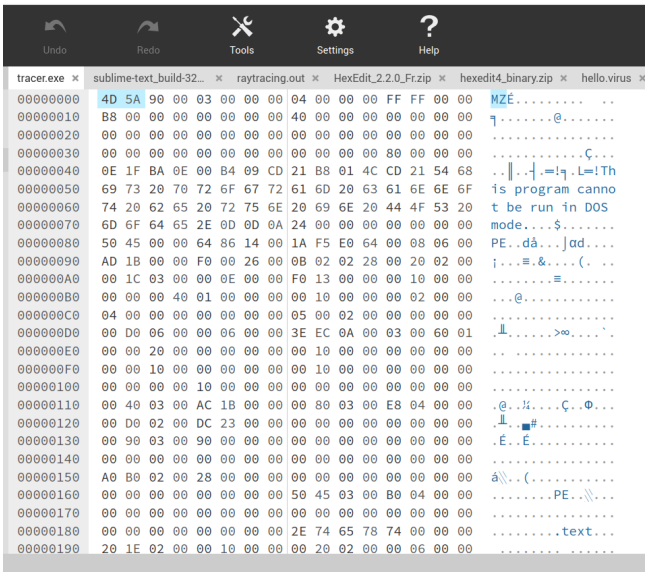

Figure 4: Exe file in Hex Editor

Then in out_or_deb_detector there is a Hex sequences for out files and deb files. And here also rule is matched if file of any of the two type are detected.

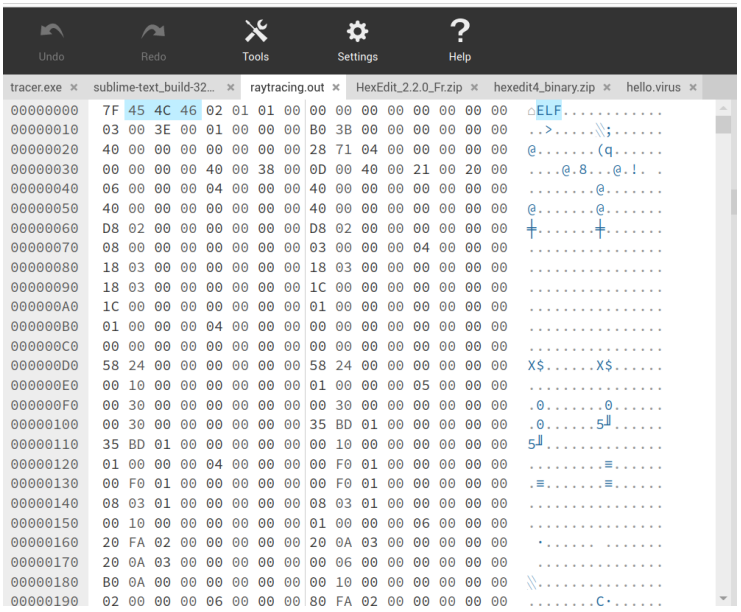

Figure 5: Out file in Hex Editor

Here is another code to match a zip file and a .out file inside a zip file.

```
24
25 rule detect_out_file_in_zip {
26     meta:
27         description = "Detects .out files inside ZIP archives"
28         date = "06-09-2023"
29     strings:
30         $zip_signature = "PK\x03\x04"
31         $out_extension = ".out"
32     condition:
33         $zip_signature at 0 and
34         any of them
35 }
36
```

Figure 6: Out file in a compressed folder detection

This is what we have in the virus folder [7].

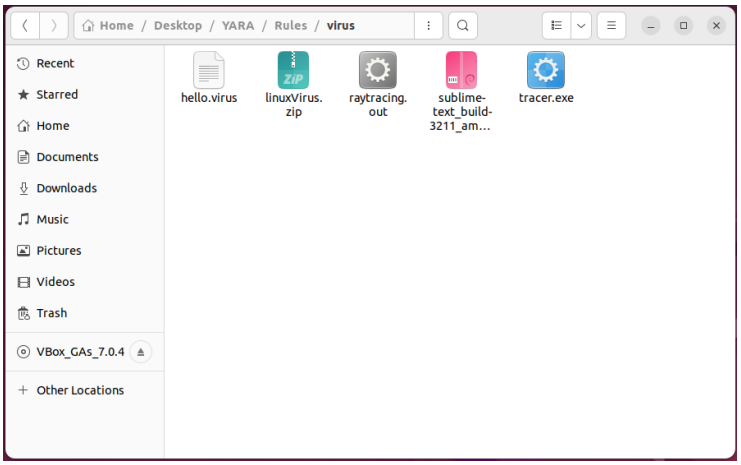

Figure 7: File structure

After running the rule file we can see all the files are detected in the folder. Also later on we can see the detection location, which rules are matched in which file in details [8].

Figure 8: Output of all the above mentioned Three Rules

4 Yara Modules

Yara provides some external modules with various functions and data structures to write mode complex rules for better malware analysis.

4.1 PE

PE modules expose the fields of the header of PE(i.e. exe) files. As a result, we can easily write rules using these functions to analyze different PE files more accurately.

4.1.1 Example:

- First, We need to write some rules related to handling PE files using PE module and save the file as "pe-demo.yara" [2]. In this file, we have two rules,
  - is_dll : if the file we are checking is a DLL(Dynamic loader linker) file.
  - is_pe : if the file we are checking is a pe(portable executable) file.

```
import "pe"

rule is_dll
{
    condition:
        pe.characteristics & pe.DLL
}

rule is_pe
{
    condition:
        pe.is_pe
}
```

Listing 2: PE Module Demo

- For demonstrating, we will download a dll file from Dll-files which should match both rules.So our current file structure looks like this [9].
- Next, we will execute our rule against this dll file using the following command [3].

```
    $yara pe-demo.yara api-ms-win-core-timezone-l1-1-0.dll
```

Listing 3: PE Module Command

- The results show which rule it matched and for which file it matched [10].

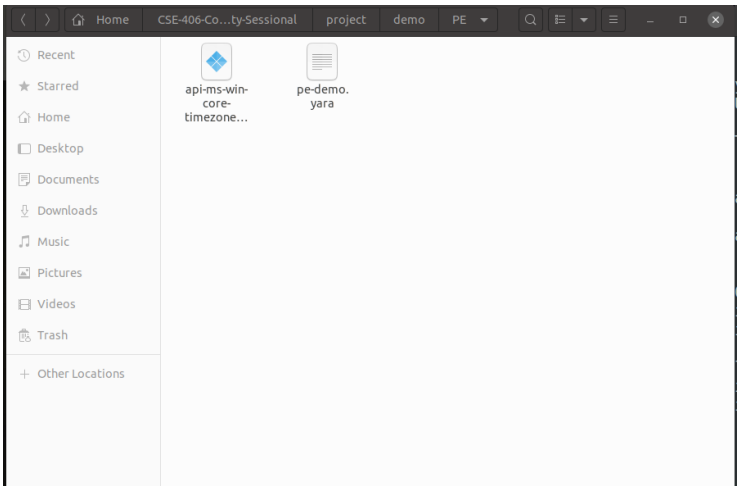

Figure 9: PE DEMO FILE STRUCTURE

Figure 10: PE DEMO RESULT

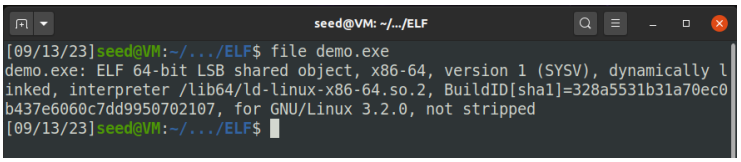

Figure 11: ELF FILE DETAILS

4.2 ELF

Similar to PE, the ELF module exposes the header of ELF files

4.2.1 Example:

- We will write rules using functions provided by ELF module and save it as elf-demo.yara [4]. The rules :
  - match_section_count : checking if the elf file header contains number of section we want to match.
  - elf_64 : if the file we are checking if the file is compatible with 64 bit machine which is written in header. The value is extracted by elf.machine and matched with integer value elf.EM_X86_64.

```
import "elf"

rule match_section_count
{
    condition:
        elf.number_of_sections == 31
}

rule elf_64
{
    condition:
        elf.machine == elf.EM_X86_64
}
```

Listing 4: ELF Module Demo

- For demonstrating, we will create an elf file "demo.exe" compatible with 64-bit system [11]. It also contains 31 sections [12]. So both rule should match for this file
- Next, we will execute our rule against this demo.exe file using the following command.

```
    $yara elf-demo.yara demo.exe
```

Listing 5: ELF Module Command

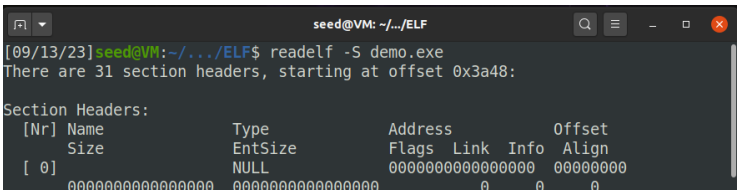

Figure 12: Elf File Section

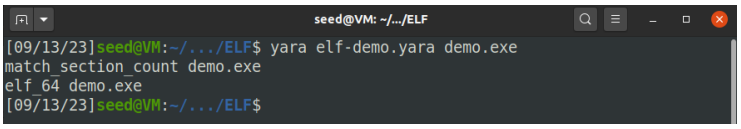

Figure 13: ELF DEMO RESULT

- The results show which rule it matched and for which file it matched. As expected, our both rules matched the demo.exe file. [13].

4.3 Cuckoo

Cuckoo enables us to write rules to check the behavior of files created by running the file in the Cuckoo sandbox. This helps us to filter out malicious files more efficiently.

4.3.1 Example:

- We will write rules using functions provided by cuckoo module and save it as cuckoo-demo.yara [6]. The rules :
  - evil_doer : This rules check if the file tries to send HTTP request to www.google.com. We can match this behavior using the function cuckoo.network.http_request provided by cuckoo module.

```
1
2 import "cuckoo"
3
4 rule evil_doer
5 {
6     condition:
7         cuckoo.network.http_request(/https:\/\/www\.google\.com/)
8 }
```

Listing 6: Cuckoo Module Demo

- For demonstrating, we will create an simple c file "evil.c" [7] which sends simple HTTP get request to google.com.Then compile the file to create evil.exe file [8].

```c
#include <stdio.h>
#include <curl/curl.h>

int main() {
    CURL *curl;
    CURLcode res;

    // Initialize libcurl
    curl = curl_easy_init();
    if (curl) {
        // Set the URL to send the GET request
        curl_easy_setopt(curl, CURLOPT_URL,
            "https://www.google.com");

        // Perform the GET request
        res = curl_easy_perform(curl);

        // Check for errors
        if (res != CURLE_OK) {
            fprintf(stderr, "curl_easy_perform() failed: %s\n",
                curl_easy_strerror(res));
        } else {
            printf("GET request to google.com was
                successful.\n");
        }

        // Clean up and free resources
        curl_easy_cleanup(curl);
    } else {
        fprintf(stderr, "Failed to initialize libcurl\n");
    }

    return 0;
}
```

Listing 7: evil.c

```
    gcc -o evil.exe evil.c -lcurl
```

Listing 8: evil.c compile Command

- Next, we have to create a behavior report file by executing the file in Cuckoo sandbox [14]
- Next, we will execute our rule against this evil.exe file using the following command [9] and download report.json.

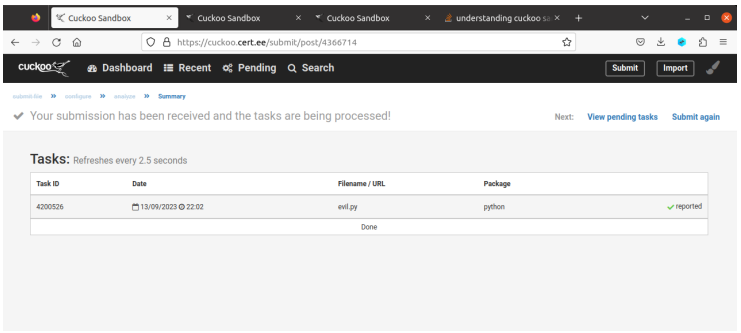

Figure 14: sandbox report generation

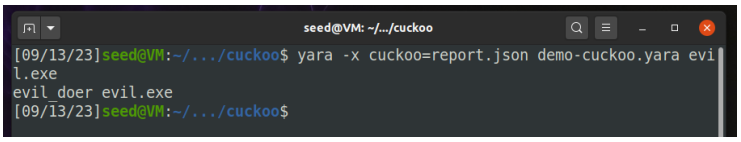

Figure 15: Cuckoo DEMO RESULT

```
$yara -x cuckoo=report.json elf-demo.yara evil.exe
```

Listing 9: Cuckoo Module Command

- The result shows us that the rule matched and indicates that evil.exe sends http_request to google.com. [15].

4.4 Hash

Hash module provides functions for handling with different hash(MD5, SHA1, SHA256).

4.4.1 Example:

- We will write rules using functions provided by hash module and save it as hash_test.yara [10]. The rules :
  - hash_demo_rule : checking if the file content from offset 0 to last character contains the string whose md5 hash or sha1 hash match with our expected string("this is hash test.") hash [16].

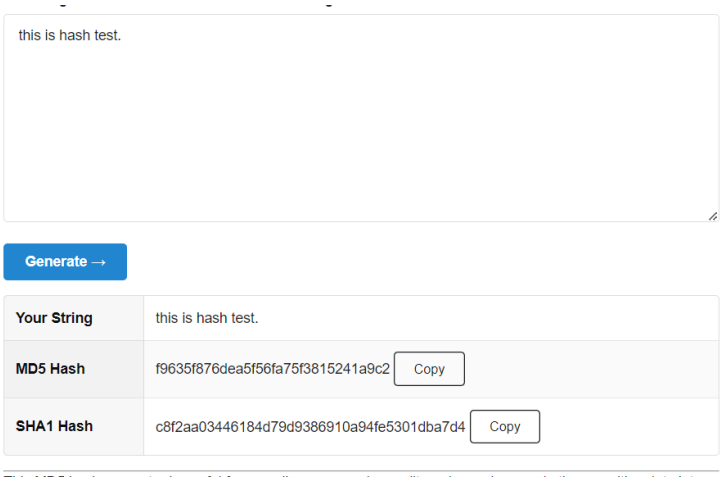

Figure 16: Hash of string

```
import "hash"

rule hash_demo_rule{
meta:
    description = "hash test"
condition:
    hash.md5(0, filesize-1)=="f9635f876dea5f56fa75f3815241a9c2"
    or
    hash.sha1(0,
        filesize-1)=="c8f2aa03446184d79d9386910a94fe5301da7d4"
}
```

Listing 10: HASH Module Demo

- For demonstrating, we will create a text hash_test.txt file containing the string "This is hash test."
- Next, we will execute our rule against this hash_test.txt file using the following command [11].

```
    $yara hash_test.yara hash_test.txt
```

Listing 11: Hash Module Command

- The results shows that our hash function matched our hash with the file[17].

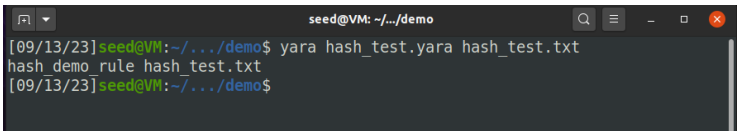

Figure 17: HASH DEMO RESULT

5 yara-python

yara-python is a python interface for Yara functionalities. It is more easy tot use and more powerful than using only Yara.

5.1 Example:

- First we will create a simple yara rule file which will only match if the file contain "test" string in it and save it as python-demo.yara [12]. We will also create a test.txt file which contains "text" string.

```
rule python_demo{
    strings:
  $a = "test"
    condition:
  $a
}
```

Listing 12: python-demo.yara

- Now, We will create a python file "python-demo.py" [13] which will use yara-pyhton module match the rule in test.txt file by first creating rule object and then calling match function.

```

import yara


rule = yara.compile('./python-demo.yara')

match = rule.match('./test.txt')

print(match)

}
```

Listing 13: python-demo.py

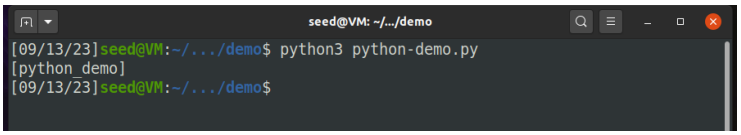

Figure 18: yara-python Result

- Next, we will simply execute the python file with this command[14].

```
$python3 python-demo.py
```

Listing 14: Yara-python Command

- The results shows that our python file executes as a simple yara file an d showing the rule that matched[18].

6 Feature Demonstration

We have recorded our feature demonstration and uploaded to Youtube which can be found in this link

7 Conclusion

Yara is a very handy tool for malware analysis with its vast modules and functionality. However, It can be easily used for various pattern-matching and file-matching task.